

TokenSense — Project Plan (MVP-Aligned)

| See what you're spending. Cut what you're wasting.

Overview

TokenSense is a B2B SaaS middleware platform that sits between users and their LLM API calls. It scores, optimizes, and tracks prompt token usage — saving companies real money with full visibility into where waste happens.

This version of the plan is built on the **MVP stack** (3 services, not 9). The product surface — scoring, optimizing, dashboard, monetization — is unchanged. What's changed is *how much infrastructure you build before you have a single paying customer*.

Core Goals

Goal	Description
Proxy	Intercept LLM API calls, log token usage, forward to provider
Score	Rate every prompt for token efficiency in real time
Optimize	Rule-based instant trim + optional AI rewrite pass
Analyze	Dashboard with history, trends, and waste leaderboards
Scale	Support individuals and enterprise orgs with RBAC
Mobile	iOS + Android app for monitoring — deferred until users ask for it

```
Frontend (Web)    → Next.js 14 + TypeScript + Tailwind CSS + shadcn/ui
Backend / Proxy   → Node.js + Hono, deployed on Railway
Primary DB        → PostgreSQL via Supabase
Auth & RBAC       → Supabase Auth (orgs + roles via Postgres tables + RLS)
DB Access Layer   → Prisma ORM
Rate Limiting     → DB-backed / in-memory counters (no Redis yet)
Analytics         → PostgreSQL (same DB) – no ClickHouse yet
Mobile App        → Deferred – responsive web dashboard covers this for now
AI Rewrite Pass    → Anthropic / OpenAI API (on-demand)
Frontend Deploy   → Vercel
Backend Deploy    → Railway
Monorepo Tool     → npm workspaces
```

What changed from the original plan, and why:

Original	Refined	Reason
Clerk	Supabase Auth	Auth lives in the same DB as everything else — no user-sync headaches, RLS gives you DB-enforced access control for free
Redis (Upstash)	DB-backed rate limiting	One fewer service; add Redis the moment rate limiting becomes a real bottleneck
ClickHouse	PostgreSQL	Postgres handles analytics fine at MVP scale; ClickHouse is a migration, not a rewrite, when volume demands it
Turborepo	npm workspaces	Zero-config monorepo tooling is enough for 3-5 packages
Cloudflare Workers	Node.js on Railway	One runtime, no edge quirks; cold-start difference is noise next to LLM response time
Expo mobile (Phase 4, weeks 13-16)	Deferred indefinitely	Don't build and maintain a second app before you know anyone wants it

Project Structure

```
tokensense/
├── apps/
│   ├── web/                # Next.js dashboard (landing + individual
+ enterprise)
│   │   ├── app/
│   │   │   ├── (landing)/
│   │   │   ├── (dashboard)/
│   │   │   └── (enterprise)/
│   │   └── components/
│   └── proxy/              # Hono middleman server (Railway)
│       ├── src/
│       │   ├── routes/    # /v1/chat, /v1/completions etc.
│       │   ├── middleware/ # Auth, logging, scoring hooks
│       │   └── forwarder.ts # Forward call to actual LLM
├── packages/
│   ├── scorer/            # Prompt efficiency scoring engine -
UNCHANGED
│   │   └── index.ts
│   ├── optimizer/        # Prompt optimization logic - UNCHANGED
│   │   ├── caveman.ts
│   │   └── ai-rewrite.ts
│   └── db/               # Prisma schema + migrations (points at
Supabase Postgres)
│       └── schema.prisma
└── package.json          # npm workspaces root, no turbo.json
```

Note: no `(apps/mobile/)`, no `(packages/analytics/)`. These get added back in when the upgrade path calls for them — the structure is designed to accept them without disruption.

Core Proxy Flow

User's App / Tool

↳ POST https://proxy.tokensense.io/v1/chat/completions

- |
- | — 1. Authenticate user via API key (validated against Supabase)
- | — 2. Score prompt (scorer package – instant)
- | — 3. Log request + token estimate to Postgres
- | — 4. Optional: Apply caveman trim or AI rewrite
- | — 5. Forward to real LLM (OpenAI / Anthropic / etc.)
- | — 6. Receive response, log actual tokens used
- | — 7. Return response to user's app

Scoring Engine (scorer package) — Unchanged

Every prompt gets a **TokenSense Score (0-100)**. Lower = more wasteful.

Flags the scorer detects:

-  Redundant context (same info repeated multiple times)
-  Bloated system prompts (boilerplate, disclaimers, filler)
-  Hedging language ("please", "could you", "I was wondering if")
-  Verbose phrasing (long sentences that say little)
-  Duplicate sentences or near-duplicate paragraphs
-  Unnecessary examples when none were asked for

Score Bands:

Score	Label	Action
85-100	 Efficient	No action needed
60-84	 Moderate Waste	Suggest caveman trim
30-59	 High Waste	Auto-flag + recommend AI rewrite
0-29	 Critical Waste	Block or force optimize (enterprise setting)

Optimizer (optimizer package) — Unchanged

1. Caveman Trim (Free, Instant, Rule-Based)

- Strips hedging words and filler phrases
- Removes duplicate sentences

- Shortens verbose clauses
- Collapses redundant context
- **Target: 20–40% token reduction, zero AI cost**

2. AI Rewrite Pass (Optional, Paid)

- Sends prompt to Claude/GPT with a compression system prompt
- Rewrites for maximum semantic density
- Preserves intent and tone
- **Target: 40–70% token reduction**

Database Design (PostgreSQL via Supabase + Prisma)

```
users           → id, email, authId (Supabase), plan, createdAt
organizations   → id, name, plan, tokenBudget, createdAt
org_members     → userId, orgId, role (admin/member/viewer)
api_keys        → id, userId, orgId, provider, encryptedKey
prompt_logs     → id, userId, orgId, promptHash, score, tokensIn,
tokensOut, cost, createdAt
labels          → id, promptLogId, name
budgets         → id, orgId, monthlyLimit, alertThreshold,
currentUsage
```

Analytics stays in this same database for now — indexed queries on `prompt_logs` (daily rollups via a scheduled query or materialized view) cover dashboard needs at MVP scale. No separate analytics store, no ETL pipeline.

Row Level Security (RLS) replaces app-level permission checks for the core rule: a user sees only their own prompts; an org admin sees their org's prompts. This is enforced in the database, not just in application code.

Auth & RBAC (Supabase Auth)

Individual Plan

- Sign up with email / Google / GitHub
- Personal API key vault
- Personal dashboard only

Enterprise Plan (Phase 3, not Phase 1)

- Org creation, roles: **Owner → Admin → Member → Viewer**
- Admins set team budgets, token limits, and alert thresholds
- RBAC controls who can use AI rewrite pass

- SSO deferred until an actual enterprise customer requests it — see Upgrade Path below
-

Dashboard Features

Individual Dashboard (Phase 2)

- ☐ Token usage over time (chart)
- ☐ Prompt history with scores
- ☐ Savings calculator (estimated \$ saved)
- ☐ Optimize any past prompt on demand
- ☐ API key management

Enterprise Dashboard (Phase 3)

- ☐ Org-wide token spend (by team, by user)
 - ☐ Waste Leaderboard (who's sending the worst prompts)
 - ☐ Budget tracker with alerts
 - ☐ RBAC management panel
 - ☐ Export usage reports (CSV)
 - ☐ Block or enforce optimization rules org-wide
-

Mobile App — Deferred

Not part of the build plan until there's demonstrated demand from real users. The responsive web dashboard covers on-the-go monitoring in the meantime. See **Upgrade Path** for what triggers building this.

Phases / Build Order

Phase 1 — Core Proxy + Scorer (Weeks 1-3)

- ☐ Set up npm workspaces monorepo
- ☐ Build Hono proxy server on Railway
- ☐ Implement scorer package (rule-based)
- ☐ Log token events to Postgres (Supabase)
- ☐ Basic auth with Supabase

Saved ~1 week vs. original plan by skipping Turborepo config and ClickHouse setup.

Phase 2 — Web Dashboard (Weeks 4-7)

- ☐ Individual dashboard (Next.js)
- ☐ Prompt history + scores
- ☐ Caveman optimizer UI

☐ API key vault UI

Phase 3 — Enterprise Features (Weeks 8–11)

- ☐ Org creation + RBAC (Supabase Auth + RLS)
- ☐ Team budgets + alerts
- ☐ Waste leaderboard
- ☐ AI rewrite pass (paid tier)

Phase 4 — Polish & Launch (Weeks 12–15)

- ☐ Landing page
- ☐ Pricing page (Individual / Pro / Enterprise)
- ☐ Onboarding flow
- ☐ Docs site (how to connect your API key)
- ☐ Beta launch

Original plan: 20 weeks to launch, including mobile. Refined plan: ~15 weeks to launch, mobile deferred to post-launch based on demand.

 Upgrade Path (build this when you have the signal, not before)

Trigger	What You Add	Effort
Rate limiting causes DB load or latency issues	Upstash Redis alongside Supabase	Isolated — swap the rate-limit check, nothing else changes
Enterprise customer requires SSO	Swap Supabase Auth → Clerk or WorkOS	Contained to the auth layer; <code>org_members</code> / <code>role</code> schema already supports it
<code>prompt_logs</code> grows past a few million rows and dashboard queries slow down	Add ClickHouse as a read-replica for analytics	Additive — Postgres stays source of truth, ClickHouse just serves reporting queries
Users ask for a mobile app	Add Expo project to the monorepo	The <code>scorer</code> / <code>optimizer</code> / <code>db</code> packages are pure TypeScript, reusable as-is
Async AI rewrite pass needs to run at scale / retries matter	Add Inngest	Wraps the existing <code>ai-rewrite.ts</code> logic, doesn't replace it
Global users need edge-latency proxy	Move Hono from Railway to Cloudflare Workers	Hono code is portable; this is a deploy-target change, not a rewrite

💰 Monetization

Plan	Target	Price (est.)
Free	Individuals	\$0 — 100 calls/day, caveman trim only
Pro	Power users	\$19/mo — unlimited calls, AI rewrite pass
Team	Small teams	\$49/mo per 5 seats, basic RBAC
Enterprise	Companies	Custom — full RBAC, SSO, budget controls, SLA

⚠️ Key Risks & Mitigations

Risk	Mitigation
Users don't trust us with their API keys	Encrypt keys at rest (AES-256), never log key values, SOC2 roadmap
Proxy adds latency	Benchmark Railway deploy, target < 50ms overhead; move to edge later if global latency becomes an issue
DB-backed rate limiting doesn't hold up under load	Watch this closely once paying customers arrive — it's the one MVP simplification most likely to need early graduation to Redis
Supabase pricing at scale	Prisma schema is portable to any Postgres host — no lock-in risk
AI rewrite pass costs money	Only trigger on explicit user request, charge accordingly

🧰 Dev Tools & DX

```
Linting           → ESLint + Prettier
Testing           → Vitest (unit tests on scorer/optimizer — the core IP)
API Docs          → Scalar (auto-generated from Hono routes)
Secret Management → Railway env vars
CI/CD             → GitHub Actions
Error Tracking    → Sentry
```

E2E testing (Playwright) deferred to post-launch, once there's a real user flow worth protecting against regressions.

Built with TokenSense. Every token counts.